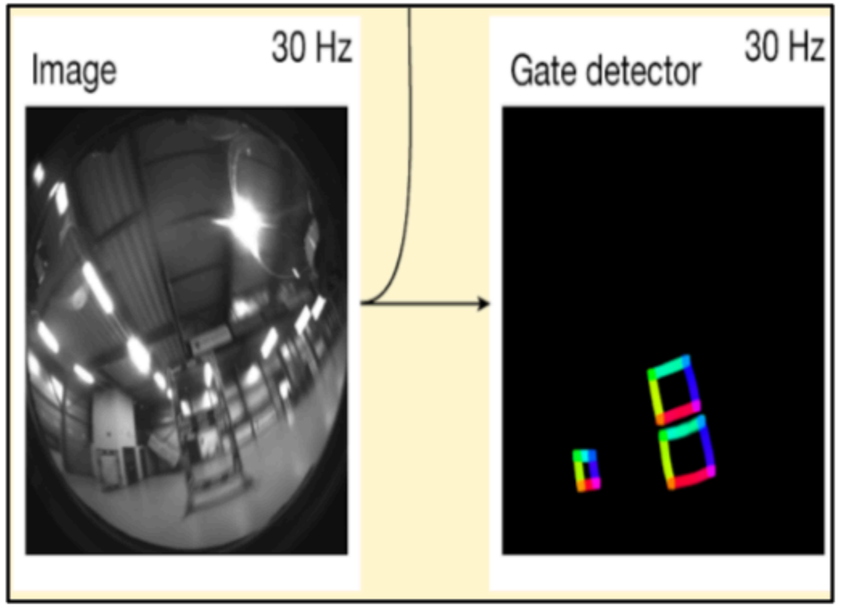



**Photo → guess → compare with label → nudge**

*Labeled images: from the simulator (next part).*

# Supervised learning

### Loss: cross-entropy

One number for how wrong a guess is. A confident wrong guess costs a lot.

### Optimization: gradient descent

Nudge the weights (the network's adjustable numbers) so the mistake shrinks.

Swift's U-Net (answers for every pixel) gives each pixel a corner probability.

Sources: Nature, 2023 (Kaufmann et al., Fig. 2, CC BY 4.0); AlphaPilot, arXiv 2005.12813

# Reinforcement learning

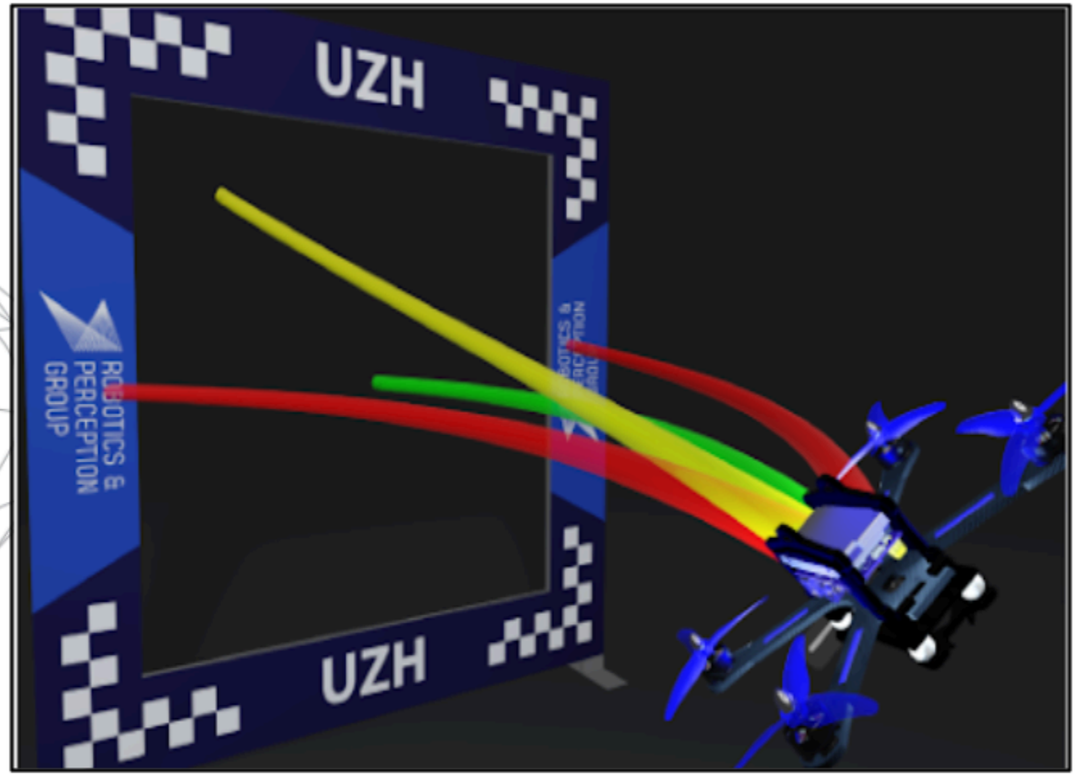
## Reward

A score after each step: progress is good, crashes are bad.

## Policy gradient

The policy is the network that picks actions. Make well-scored actions likelier.

PPO (Proximal Policy Optimization) keeps every update small, so one bad step cannot ruin the policy.




**Fly → score → do more of what worked**

*Simulated flights, scored by the reward*

Sources: Nature, 2023 (Kaufmann et al., Fig. 2b, CC BY 4.0); PPO, arXiv 1707.06347

# Imitation learning

Learn by copying an expert. No score (reinforcement) and no labelled photo (supervised): the example is what the expert did.



Barrel roll, power loop, Matty flip: trained only in simulation

**Teacher: the expert. It sees the exact state (simulation only)**

↓

**Student: sees only the camera and copies the teacher's actions**

↓

**DAgger: the student flies, the teacher corrects where it drifted**

*Training data = what the expert does in each situation*

## Invent or copy?

Should a machine invent its own strategy, or copy a human?

Sources: Nature, 2023 (Kaufmann et al.); Deep Drone Acrobatics, RSS 2020 (arXiv 2006.05768); DAgger (arXiv 1011.0686)

# Learning in Swift




Imitation learning is not part of Swift. The same lab used it for acrobatic flips, with a teacher that sees the exact state in simulation.

Sources: Nature, 2023 (Kaufmann et al.); Deep Drone Acrobatics, RSS 2020 (arXiv 2006.05768)